Final Project Requirements, Questions and Architecture

Final Project Definition

The project you are designing here is **your** final project — either one of the ideas on the **Final Project Specifications** page, or a project idea the instructor has approved. Read that page end to end (overview, required showcase sections, rubric, and the idea you picked) before you start this exercise.

Assignment Overview

Anyone can build an agentic system with Claude Code or Open Code. *Can you build one that is reliable, safe, cost effective and meets all the requirements every time it's run?* It's time to put on your AI architect hat and address LLM/RAG shortcomings with a good agentic AI design.

Rubric

This is a 500 point exercise. It's an assessment of how much you have learned up to this point. Please take it seriously. It's as important as the project output. You should complete this Quarto markdown file (.qmd) and turn it in along (as .qmd) with a quarto rendered PDF version (.pdf). Add `format: pdf` to the yaml header of this file and then run `quarto render yourfilename.qmd` to build the PDF.

Important Note

This work must be done 100% without assistance from any AI. I need to know that *YOU* have learned the material. I know that Claude knows it. **Any use of AI will be easily detected and will be treated as an Honor Code violation. Please take this seriously.**

Introduction

Unlike the mid-term, there is no single specification for everyone — so before you list requirements, tell me what you are building. Name the project you chose: one of the ideas on the Final Project Specifications page, **or** a project the instructor has approved (say so, and when it was approved).

Then write a few paragraphs (in prose) explaining what you are trying to accomplish. Who is the user? What problem do they have today, and what will they be able to do with your solution that they cannot do now? What is in scope and — just as important — what is out of scope? What does a successful run look like? Finally, make the case for why this problem **needs a multi-agent design** rather than a single prompt or a script.

The project I have chosen is Student Scholarship Matcher (approved by the instructor on 2026-10-27).

The project is on student-scholarship matching and was approved Oct 27, 2026

The goal of this project is to help match students to scholarships they qualify for and can reasonably get while also meeting student requirements for scholarships such as no-essay.

The first reason for this system is that financial aid has been decreasing for students, causing many to reconsider the college they plan to go to, they are going to, or going to college altogether. The second reason is there are tens of thousands of scholarships and it is suffocating for students to dig through them while also balancing their senior year of high school or there current college work.

This project should save hours of a students time while also relieving them of the stress and anxiety of scholarship searching and affording college.

The current vision of this project is that it will take the information of the student: location, year, gpa, clubs, interests, past essays, major/major of interest, financial qualification (Pell status), past scholarships granted. It will then find scholarships that the students qualify for letting the student choose one to go into and fill out.

This will begin by being scoped to Virgina Tech and the schools my sister wants to go to meaning it is scoped to VA, NC, and SC.

Requirements

List the requirements for **your** project, based on the Final Project Specifications (the overview, required showcase sections, rubric, and the description of the idea you chose — or your instructor-approved proposal) and on the design you are proposing in your Introduction. List as many as you can (20 is not meaningful here). This is a critical step! Any person or LLM should be able to fully understand **all** requirements for the application from this list.

Number every requirement — the later sections refer back to them by number.

1. Must scrape scholarships from the list of list of scholarship sites (scrap.md has the list).
2. Must have an interface for students to enter their information such as age, location, gpa, school status, financial status, gender, race, disability, , as well as files that are past essays.
3. Must allow the student to save interested scholarships and retrieve them later
4. Must retrieve the scholarship as well as the open and close date and where the scholarship is from.
5. Must retrieve scholarships the student qualifies for based on the information they provide
6. Must store information a student enters and allow them to edit it in the future.
7. Must filter out scam and spam scholarships like the gambling ones
8. Must extract hard and soft criteria, as well as scholarship application requirements like essays or recommendations.
9. Must consider how "easy" the scholarship is as well as the probability of getting it based on number of awards, local, regional, national, application requirements
10. Must source scholarship matches in the student's profile and justify level of difficulty based on R9. Must be defensible, auditable, and consistent.

11. Must store scraped scholarships and unlist ones that passed the application deadline but retain them in the database.
12. Must have a chat interface with the student for follow up discussion on a scholarship
13. Must have a notification if there are no scholarships for the student.
14. Must allow users to rank the scholarships retrieved based on student controlled ordering (highest/lowest probability, easiest/hardest, least/most reward, highest/lowest expected value).
15. Must filter scholarships deterministically before going to LLM review
16. Must have a clear rubric for each scholarship recommended for student.

LLM / RAG Issues

Referring back to the lectures on LLM and RAG — and, now that you have seen them, on multi-agent systems and anti-patterns — what are the specific issues that you must overcome with good agentic AI design/architecture/patterns **to meet the requirements**? **Make sure you point back to a specific requirement**. Don't solve the issue yet. Just identify it. You should have a fair number of these (between 5 and 20).

1. (Requirement 2) The LLM would get access to personal student information
2. (Requirement 5) The LLM may hallucinate that scholarships do or don't match with the student
3. (Requirement 4, 8) LLM tagging of key information for each scholarship could miss this info. This could be due to a hallucination or even context rot making it lose its direction through reading long scholarships.
4. (Requirement 9, 10) The LLM will have to try to objectively score scholarships based on a rubric and give a rough estimation of the chance of a student getting a scholarship. These will be holistic and incalculable meaning that LLM non-determinism and hallucination will affect the grading and prevent it from being 100% consistent.
5. (Requirement 12) The LLM must be stateful and not reach context rot after generating the team, so that it can discuss with the user. It also needs to be able to remember its reasoning and not hallucinate.

Basic Architecture

Write up a couple of paragraphs (in prose) on how you plan to design the solution. What will be the general approach. Don't solve the issues yet… just general approach. What are the major pieces, how do they connect, etc.?

The solution will start with the student following an interface that asks information about the student. Then a deterministic SQL filtering will filter scholarships based on student information like GPA hard requirements for scholarships and such. Then a Supervisor sends the remiaing scholarships to an LLM that filters further based on more fuzzy student content and filters the scholarships further. When the supervisor thinks that it filtered well it sends the scholarships to an LLM scorer who scores and justifies the score for each scholarship. One the supervisor thinks the scoring is good and well justified based on the rubric then the remaining scholarships are sent

back to the user in a table interface where they can filter them based on things like award amount and chance of winning, and they can select scholarships and ask follow up questions with an LLM.

25 Bonus points for including a UML sequence diagram using a mermaid diagram. For this syntax (and only this syntax), I will allow LLM usage** but you need to prompt the LLM with the sequence diagram flow, actors and steps. Include your prompt here.**

Prompt used to generate the sequence diagram syntax:

make this Sequence diagram in mermaid for a student scholarship matcher: 1. User enters info about them like gpa and such, and also any hobbies and such. 2. INformation matched against schoalrship database and filtered using SQL. 3. Fitlered scholarships sent to Supervisor who sends scholarships to LLM filtering for fuzzy infroamtion 4. Looping with Supervisor until sufficient 5. Supervisor sends LLM filtered scholarships to LLM scorer 6. Loops with supervisor until scores are well justificed 7. Final scholarships sent as a filterable table with the user with each being selectable and LLM chatbot follow ups with User. 4. Looping with Supervisor until sufficient 5. Supervisor sends LLM filtered scholarships to LLM scorer 6. Loops with supervisor until scores are well justificed 7. Final scholarships sent as a filterable table with the user with each being selectable and LLM chatbot follow ups with User.

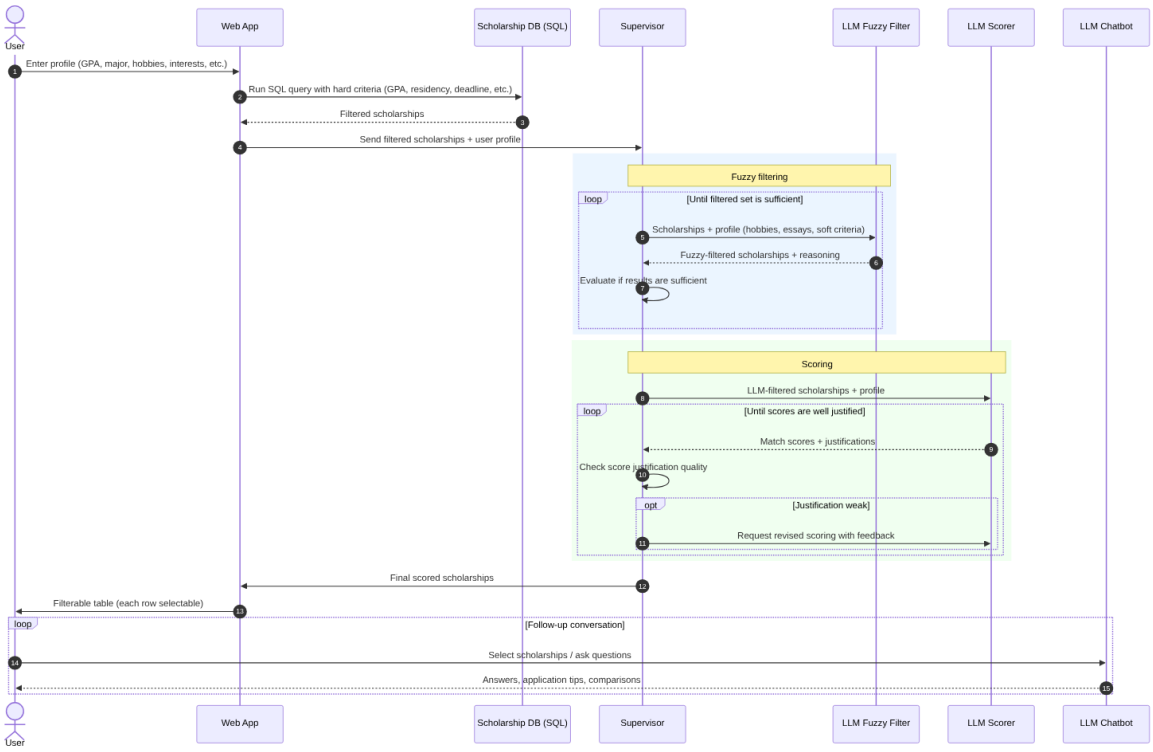


Figure 1: Sequence diagram

You should end your `Basic Architecture` with the level of agency you think is **earned** by this solution architecture. Go back to the beginning of the course for the various levels. **Make sure you give a quick summary of why it is earned.**

The level of agency earned is Multiagent This is because there is a baton pass of tasks and there needs to be a manager that says this work looks good before the next step. It's like having a boss

check the work of his subordinate before sending it to the client.

Solution Mitigation

Now for each of the LLM/RAG issues, you should give specific design patterns that we learned in class to address them. I need a short explanation of why your solution is the right one. **Make sure you refer back to the numbered issue from the previous section.**

1. (Issue 1) Example pattern name. The solution will use the ARC LLMs which are cleared for high risk data.
2. (Issue 2) Chain-of-verification run against the LLM retriever by the supervisor LLM. This works best because the supervisor has to approve the work already.
3. (Issue 3) Reflexion for each scholarship is the correct approach because there are thousands of scholarships and not much time and cost can be spent on each one, so a cheap check works best.
4. (Issue 4) Reflexion for the scoring LLM + Chain of Verification with the Supervisor when it checks the scoring LLM's work. This works because the scoring LLM can double check its work and find mistakes before having to pass it back to the supervisor and the supervisor needs a way to systematically check the scoring LLM's work.
5. (Issue 5) Compacting + Saved conversations + ReAct Loop. By saving conversations the LLM can reference information from them in future conversations and by compacting the LLM can continue the same conversation. By having the LLM ReAct over the conversation it can check and ground its answers from previous parts of the conversation.

Final Thoughts

Just like the mid-term, this exercise is something you do before invoking an agentic harness. This is where you add value and earn your place in the future AI ecosystem. Next steps we will review this architecture with a frontier level model and refine it. Then we work with an agentic harness to build it!